Can Applications Recover from `fsync` Failures?

Anthony Rebello, Yuvraj Patel, Ramnatthan Alagappan,
Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau
Computer Sciences Department, University of Wisconsin – Madison

Abstract

We analyze how file systems and modern data-intensive applications react to `fsync` failures. First, we characterize how three Linux file systems (ext4, XFS, Btrfs) behave in the presence of failures. We find commonalities across file systems (pages are always marked clean, certain block writes always lead to unavailability), as well as differences (page content and failure reporting is varied). Next, we study how five widely used applications (PostgreSQL, LMDB, LevelDB, SQLite, Redis) handle `fsync` failures. Our findings show that although applications use many failure-handling strategies, none are sufficient: `fsync` failures can cause catastrophic outcomes such as data loss and corruption. Our findings have strong implications for the design of file systems and applications that intend to provide strong durability guarantees.

1 Introduction

Applications that care about data must care about how data is written to stable storage. Issuing a series of `write` system calls is insufficient. A `write` call only transfers data from application memory into the operating system; the OS usually writes this data to disk lazily, improving performance via batching, scheduling, and other techniques [25, 44, 52, 53].

To update persistent data correctly in the presence of failures, the order and timing of flushes to stable storage must be controlled by the application. Such control is usually made available to applications in the form of calls to `fsync` [9, 47], which forces unwritten ("dirty") data to disk before returning control to the application. Most update protocols, such as write-ahead logging or copy-on-write, rely on forcing data to disk in particular orders for correctness [30, 31, 35, 38, 46, 56].

Unfortunately, recent work has shown that the behavior of `fsync` during failure events is ill-defined [55] and error prone. Some systems, for example, mark the relevant pages clean upon `fsync` failure, even though the dirty pages have not yet been written properly to disk. Simple application responses, such as retrying the failed `fsync`, will not work as expected, leading to potential data corruption or loss.

In this paper, we ask and answer two questions related to this critical problem. The first question (§3) relates to the file system itself: why does `fsync` sometimes fail, and what is the effect on file-system state after the failure event?

To answer this first question, we run carefully-crafted micro-workloads on important and popular Linux file systems (ext4 [43], XFS [54], Btrfs [50]) and inject targeted block failures in the I/O stream. We then use a combination of tools to examine the results. Our findings show commonalities across file systems as well as differences. For example, all three file systems mark pages clean after `fsync` fails, rendering techniques such as application-level retry ineffective. However, the content in said clean pages varies depending on the file system; ext4 and XFS contain the latest copy in memory while Btrfs reverts to the previous consistent state. Failure reporting is varied across file systems; for example, ext4 data mode does not report an `fsync` failure immediately in some cases, instead (oddly) failing the subsequent call. Failed updates to some structures (e.g., journal blocks) during `fsync` reliably lead to file-system unavailability. And finally, other potentially useful behaviors are missing; for example, none of the file systems alert the user to run a file-system checker after the failure.

The second question we ask is (§4): how do important data-intensive applications react to `fsync` failures? To answer this question, we build CuttleFS, a FUSE file system that can emulate different file system `fsync` failures. CuttleFS maintains its own page cache in user-space memory, separate from the kernel page cache, allowing application developers to perform durability tests against characteristics of different file systems, without interference from the underlying file system and kernel.

With this test infrastructure, we examine the behavior of five widely-used data-management applications: Redis [18], LMDB [15], LevelDB [12], SQLite [20] (in both RollBack [1] and WAL modes [21]), and PostgreSQL [15] (in default and DirectIO modes). Our findings, once again, contain both specifics per system, as well as general results true across some or all. Some applications (Redis) are surprisingly careless with `fsync`, not even checking its return code before returning success to the application-level update; the result is a database with old, corrupt, or missing keys. Other applications (LMDB) exhibit false-failure reporting, returning an error to users even though on-disk state is correct. Many applications (Redis, LMDB, LevelDB, SQLite) exhibit data corruptions; for example, SQLite fails to write data to its rollback journal and corrupts in-memory state by reading from said journal when a transaction needs to be rolled back. While corruptions can cause some applications to reject newly inserted records (Redis, LevelDB, SQLite), both new and old data can be lost on updates (PostgreSQL). Finally, applications (LevelDB, SQLite, PostgreSQL) sometimes seemingly

work correctly as long as the relevant data remains in the file-system cache; when said data is purged from the cache (due to cache pressure or OS restart), however, the application then returns stale data (as retrieved from disk).

We also draw high-level conclusions that take both file-system and application behavior into account. We find that applications expect file systems on an OS platform (e.g., Linux) to behave similarly, and yet file systems exhibit nuanced and important differences. We also find that applications employ numerous different techniques for handling `fsync` failures, and yet none are (as of today) sufficient; even after the PostgreSQL `fsync` problem was reported [55], no application yet handles its failure perfectly. We also determine that application recovery techniques often rely upon the file-system page cache, which does not reflect the persistent state of the system and can lead to data loss or corruption; applications should ensure recovery protocols only use existing persistent (on-disk) state to recover. Finally, in comparing ext4 and XFS (journaling file systems) with Btrfs (copy-on-write file system), we find that the copy-on-write strategy seems to be more robust against corruptions, reverting to older states when needed.

The rest of this paper is organized as follows. First, we motivate why this study is necessary (§2), followed by a file-system study (§3). Next, we study how applications react to `fsync` failures (§4). We then discuss the implications of our findings (§5), discuss related work (§6), and conclude (§7).

2 Motivation

Applications that manage data must ensure that they can handle and recover from any fault that occurs in the storage stack. Recently, a PostgreSQL user encountered data corruption after a storage error and PostgreSQL played a part in that corruption [17]. Because of the importance and complexity of this error, we describe the situation in detail.

PostgreSQL is an RDBMS that stores tables in separate files and uses a write-ahead log *(wal)* to ensure data integrity [16]. On a transaction commit, the entry is written to the log and the user is notified of the success. To ensure that the log does not grow too large (as it increases startup time to replay all entries in the log), PostgreSQL periodically runs a checkpoint operation to flush all changes from the log to the different files on disk. After an `fsync` is called on each of the files, and PostgreSQL is notified that everything was persisted successfully, the log is truncated.

Of course, operations on persistent storage do not always complete successfully. Storage devices can exhibit many different types of partial and transient failures, such as latent sector errors [27, 41, 51], corruptions [26], and misdirected writes [42]. These device faults are propagated through the file system to applications in a variety of ways [40, 49], often causing system calls such as `read`, `write`, and `fsync` to fail with a simple return code.

When PostgreSQL was notified that `fsync` failed, it retried the failed `fsync`. Unfortunately, the semantics for what should happen when a failed `fsync` is retried are not well defined. While POSIX aims to standardize behavior, it only states that outstanding IO operations are not guaranteed to have been completed in the event of failures during `fsync` [14]. As we shall see, on many Linux file systems, data pages that fail to be written, are simply marked clean in the page cache when `fsync` is called and fails. As a result, when PostgreSQL retried the `fsync` a second time, there were no dirty pages for the file system to write, resulting in the second `fsync` succeeding without actually writing data to disk. PostgreSQL assumed that the second `fsync` persisted data and continued to truncate the write-ahead log, thereby losing data. PostgreSQL had been using `fsync` incorrectly for 20 years [55].

After identifying this intricate problem, developers changed PostgreSQL to respond to the `fsync` error by crashing and restarting without retrying the `fsync`. Thus, on restart, PostgreSQL rebuilds state by reading from the *wal* and retrying the entire checkpoint process. The hope and intention is that this crash and restart approach will not lose data. Many other applications like WiredTiger/MongoDB [24] and MySQL [3] followed suit in fixing their `fsync` retry logic.

This experience leads us to ask a number of questions. As application developers are not certain about the underlying file-system state on `fsync` failure, the first part of our study answers what happens when `fsync` fails. How do file systems behave after they report that an `fsync` has failed? Do different Linux file systems behave in the same way? What can application developers assume about the state of their data after an `fsync` fails? Thus, we perform an in-depth study into the `fsync` operation for multiple file systems.

The second part of our study looks at how data-intensive applications react to `fsync` failures. Does the PostgreSQL solution indeed work under all circumstances and on all file systems? How do other data-intensive applications react to `fsync` failures? For example, do they retry a failed `fsync`, avoid relying on the page cache, crash and restart, or employ a different failure-handling technique? Overall, how well do applications handle `fsync` failures across diverse file systems?

3 File System Study

Our first study explores how file systems behave after reporting that an `fsync` call has failed. After giving a brief background of caching in file systems, we describe our methodology and our findings for the three Linux file systems.

3.1 Background

File systems provide applications with `open`, `read`, and `write` system calls to interact with the underlying storage media. Since block devices such as hard disks and solid state drives are much slower than main memory [57], the operating system maintains a page cache of frequently used pages of files in kernel space in main memory.

When an application calls `read`, the kernel first checks if the data is in the page cache. If not, the file system retrieves

the data from the underlying storage device and stores it in the page cache. When an application calls `write`, the kernel only *dirties* the page in memory while notifying the application that the `write` succeeded; there is now a mismatch between the data in memory and on the device and data can potentially be lost. For durability, the file system periodically synchronizes content between memory and disk by *flushing* dirty pages and marking them clean. Applications that require stronger durability guarantees can force the dirty pages to disk using the `fsync` system call.

Applications can choose to bypass the page cache altogether by opening files with *O_DIRECT* (DirectIO). For caching, applications must perform their own in user space. Calls to `fsync` are still required since data may be cached within the underlying storage media; an `fsync` issues a FLUSH command to the underlying device so it pushes data all the way to stable storage.

3.2 Methodology

To understand how file systems should behave after reporting an `fsync` failure, we begin with the available documentation. The `fsync` man pages [9] report that `fsync` may fail for many reasons: the underlying storage medium has insufficient space (*ENOSPC* or *EDQUOT*), the file descriptor is not valid (*EBADF*), or the file descriptor is bound to a file that does not support synchronization (*EINVAL*). Since these errors can be discovered by validating input and metadata before initiating write operations, we do not investigate them further.

We focus on errors that are encountered only after the file system starts synchronizing dirty pages to disk; in this case, `fsync` signals an *EIO* error. *EIO* errors are difficult to handle because the file system may have already begun an operation (or changed state) that it may or may not be able to revert.

To trigger *EIO* errors, we consider single, transient, write faults in line with the fail-partial failure model [48,49]. When the file system sends a write request to the storage device, we inject a fault for a single sector or block within the request. Specifically, we build a kernel module device-mapper target that intercepts block-device requests from the file system and fails a particular write request to a particular sector or block while letting all other requests succeed; this allows us to observe the impact on an unmodified file system.

3.2.1 Workloads

To exercise the `fsync` path, we create two simple workloads that are representative of common write patterns seen in data-intensive applications.

Single Block Update (w_{su}): open an existing file containing three pages (12KB) and modify the middle page. This workload resembles many applications that modify the contents of existing files: LMDB always modifies the first two metadata pages of its database file; PostgreSQL stores tables as files on disk and modifies them in-place. Specifically, w_{su} issues system calls in the following sequence: `open`, `lseek(4K)`, `write(4K)`, *`fsync`*, *`fsync`*, `sleep(40)`, `close`. The first `fsync` forces the dirty page to disk. While one `fsync` is sufficient in the absence of failures, we are interested in the impact of `fsync` retries after a failure; therefore, w_{su} includes a second `fsync`. Finally, since ext4, XFS, and Btrfs write out metadata and checkpoint the journal periodically, w_{su} includes a sleep for 40 seconds.

Multi Block Append (w_{ma}): open a file in append mode and write a page followed by an `fsync`; writing and fsyncing is repeated after sleeping. This workload resembles many applications that periodically write to a log file: Redis writes every operation that modifies its in-memory data structures to an append only file; LevelDB, PostgreSQL, and SQLite write to a write-ahead-log and `fsync` the file after the write. w_{ma} repeats these operations after a delay to allow checkpointing to occur; this is realistic as clients do not always write continuously and checkpointing may occur in those gaps. Specifically, w_{ma} issues system calls in the following sequence: `open` (in append mode), `write(4K)`, *`fsync`*, `sleep(40)`, `write(4K)`, *`fsync`*, `sleep(40)`, `close`.

3.2.2 Experiment Overview

We run the workloads on three different file systems: ext4, XFS, and Btrfs, with default mkfs and mount options. We evaluate both ext4 with metadata ordered journaling (data=ordered) and full data journaling (data=journal). We use an Ubuntu OS with Linux kernel version 5.2.11.

For each file system and workload, we first trace the block write access pattern. We then repeat the workload multiple times, each time configuring the fault injector to fail the ith write access to a given sector or block. We only fail a single block or sector within the block in each iteration. We use a combination of offline tools (debugfs and xfs_db) and documentation to map each block to its respective file system data structure. We use SystemTap [22] to examine the state of relevant buffer heads and pages associated with data or metadata in the file system.

3.2.3 Behavior Inference

We answer the following questions for each file system:

Basics of `fsync` Failures:

- Q1 Which block (data, metadata, journal) failures lead to `fsync` failures?
- Q2 Is metadata persisted if a data block fails?
- Q3 Does the file system retry failed block writes?
- Q4 Are failed data blocks marked clean or dirty in memory?
- Q5 Does in-memory page content match what is on disk?

Failure Reporting:

- Q6 Which future `fsync` will report a write failure?
- Q7 Is a write failure logged in the syslog?

After Effects of `fsync` Failure:

- Q8 Which block failures lead to file-system unavailability?

Q9 How does unavailability manifest? Does the file system shutdown, crash, or remount in read-only mode?

Q10 Does the file suffer from holes or block overwrite failures? If so, in which parts of a file can they occur?[1]

Recovery:

Q11 If there is any inconsistency introduced due to `fsync` failure, can fsck detect and fix it?

3.3 Findings

We now describe our findings for the three file systems we have characterized: ext4, XFS, and Btrfs. Our answers to our posed questions are summarized in Table 1.

3.3.1 Ext4

The ext4 file system is a commonly-used journaling file system on Linux. The two most common options when mounting this file system are *data=ordered* and *data=journal* which enable ext4 ordered mode and ext4 data mode, respectively. Ext4 ordered mode writes metadata to the journal whereas ext4 data mode writes both data and metadata to the journal.

Ext4 ordered mode: We give an overview of ext4 ordered mode by describing how it behaves for our two representative workloads when no failures occur.

Single Block Update (w_{su}). When no fault is injected and `fsync` is successful, ext4 ordered mode behaves as follows. During the `write` (Step 1), ext4 updates the page in the page cache with the new contents and marks the page dirty. On `fsync`, the page is written to a data block; after the data-block write completes successfully, the metadata (i.e., the inode with a new modification time) is written to the journal, and `fsync` returns 0 indicating success (Step 2). After the `fsync`, the dirty page is marked clean and contains the newly written data. On the second `fsync`, as there are no dirty pages, no block writes occur, and as there are no errors, `fsync` returns 0 (Step 3). During `sleep`, the metadata in the journal is checkpointed to its final in-place block location (Step 4). No writes or changes in page state occur during the `close` (Step 5).

If `fsync` fails (i.e., returns -1 with errno set to *EIO*), a variety of write problems could have occurred. For example, the data-block write could have failed; if this happens, ext4 does not write the metadata to the journal. However, the updated page is still marked clean and contains the newly written data from Step 1, causing a discrepancy with the contents on disk. Furthermore, even though the inode table was not written to the journal at the time of the data fault, the inode table containing the updated modification time is written to the journal on the second `fsync` in Step 3. Steps 4 and 5 are the same as above, and thus the inode table is checkpointed.

Thus, applications that read this data block while the page remains in the page cache (i.e., the page has not been evicted and the OS has not been rebooted) will see the new contents of the data; however, when the page is no longer in memory and must be read from disk, applications will see the old contents.

Alternatively, if `fsync` failed, it could be because a write to one of the journal blocks failed. In this case, ext4 aborts the journal transaction and remounts the file system in read-only mode, causing all future `writes` to fail.

Multi Block Append (w_{ma}). This next workload exercises additional cases in the `fsync` error path. If there are no errors and all `fsyncs` are successful, the multi-block append workload on ext4 behaves as follows. First, during `write`, ext4 creates a new page with the new contents and marks it dirty (Step 1). On `fsync`, the page is written to a newly allocated on-disk data block; after the data-block write completes successfully, the relevant metadata (i.e., both the inode table and the block bitmap) are written to the journal, and `fsync` returns success (Step 2). As in w_{su}, the page is marked clean and contains the newly written data. During `sleep`, the metadata is checkpointed to disk (Step 3); specifically, the inode contains the new modification time and a link to the newly allocated block, and the block bitmap now indicates that the newly allocated block is in use. The pattern is repeated for the second `write` (Step 4), `fsync` (Step 5), and `sleep` (Step 6). As in w_{su}, there are no write requests or changes in page state during `close` (Step 7).

An `fsync` failure could again indicate numerous problems. First, a write to a data block could have failed in Step 2. If this is the case, the `fsync` fails and the page is marked clean; as in w_{su}, the page contains the newly written data, differing from the on-disk block that contains the original block contents. The inode table and block bitmap are written to disk in Step 3; thus, even though the data itself has not been written, the inode is modified to reference this block and the corresponding bit is set in the block bitmap. When the workload writes another 4KB of data in Step 4, this write continues oblivious of the previous fault and Steps 5, 6, and 7 proceed as usual.

Thus, with a data-block failure, the on-disk file contains a non-overwritten block where it was supposed to contain the data from Step 1. A similar possibility is that the write to a data block in Step 5 fails; in this case, the file has a non-overwritten block at the end instead of somewhere in the middle. Again, an application that reads any of these failed data blocks while they remain in the page cache will see the newly appended contents; however, when any of those pages are no longer in memory and must be read from disk, applications will read the original block contents.

An `fsync` failure could also indicate that a write to a journal-block failed. In this case, as in w_{su}, the `fsync` returns an error and the following `write` fails since ext4 has been remounted in read-only mode.

Because this workload contains an `fsync` after the metadata has been checkpointed in Step 3, it also illustrates the impact of faults when checkpointing the inode table and block bitmap. We find that despite the fact that a write has failed and

[1] In file-system terminology, a hole is a region in a file for which there is no block allocated. If a block is allocated but not overwritten with the new data, we consider the file to have a *non-overwritten block* and suffer from *block overwrite failure*.

		fsync Failure Basics					Error Reporting		After Effects			Recovery
		Which block failure causes fsync failure?	Is metadata persisted on data block failure?	Which block failures are retried?	Is the page dirty or clean after failure?	Does the in-memory content match disk?	Which fsync reports the failure?	Is the failure logged to syslog?	Which block failure causes unavailability?	What type of unavailability?	Holes or block over-write failures? If yes where do they occur?	Can fsck help detect holes or block over-write failures?
		Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10	Q11
ext4	ordered	data,jrnl	yes [A]		clean [B]	no [B]	immediate	yes	jrnl	remount-ro	NOB, anywhere [A]	no
	data	data,jrnl	yes [A]		clean [B]	no [B]	next [C]	yes	jrnl	remount-ro	NOB, anywhere [A]	no
XFS		data,jrnl	yes [A]	meta	clean [B]	no [B]	immediate	yes	jrnl,meta	shutdown	NOB, within [A]	no
Btrfs		data,jrnl	no		clean	yes	immediate	yes	jrnl,meta	remount-ro	HOLE, within [D]	yes

[A] Non-overwritten blocks (Q10) occur because metadata is persisted despite data-block failure (Q2).
[B] Marking a dirty page clean (Q4) even though the content does not match the disk (Q5) is problematic.
[C] Delayed reporting (Q6) of fsync failures may confuse application error-handling logic.
[D] Continuing to write to a file after an fsync failure is similar to writing to an offset greater than file size, causing a hole in the skipped portion (Q10).

Table 1: **Behavior of Various File Systems when fsync Fails.** *The table summarizes the behavior of the three file systems: ext4, XFS, and Btrfs according to the questions posed in Section 3.2.3. The questions are divided into four categories mentioned at the top. For questions that require identifying a block type, we use the following abbreviations: Data Block (data), Journal Block (jrnl), Metadata Block (meta). In Q9, Remount-**ro** denotes remounting in read-only mode. In Q10, "anywhere" and "within" describe the locations of the holes or non-overwritten blocks (NOB); "within" does not include the end of the file. Entries with a superscript denote a problem.*

the file system will now be in an inconsistent state, the following fsync does not return an error. However, the metadata error is logged to syslog.

We note that for none of these fsync failures does ext4 ordered mode recommend running the file system checker; furthermore, running the checker does not identify or repair any of the preceding problems. Finally, future calls to fsync never retry previous data writes that may have failed. These results for ext4 ordered mode are all summarized in Table 1.

The ext4 file system also offers functionality to abort the journal if an error occurs in a file data buffer (mount option data_err=abort) and remount the file system in read-only mode on an error (mount option errors=remount-ro). However, we observe that the results are identical with and without the mount options. [2]

Ext4 Data Mode: Ext4 data mode differs from ordered mode in that data blocks are first written to the journal and then later checkpointed to their final in-place block locations.

As shown in Table 1, the behavior of fsync in ext4 data mode is similar to that in ext4 ordered mode for most cases: for example, on a write error, pages may be marked clean even if they were not written out to disk, the file system is remounted in read-only mode on journal failures, meta-data failures are not reported by fsync, and files can end up with non-overwritten blocks in the middle or end.

However, the behavior of ext4 data mode differs in one important scenario. Because data blocks are first written to the journal and later to their actual block locations during checkpointing, the first fsync after a write may succeed even if a data block will not be successfully written to its permanent in-place location. As a result, a data-block fault causes the second fsync to fail instead of the first; in other words, the error reporting by fsync is delayed due to a *failed intention* [36].

[2] We verified our observations by reproducing them using standard Linux tools and have filed a bug report for the same [2].

3.3.2 XFS

XFS is a journaling file system that uses B-trees. Instead of performing physical journaling like ext4, XFS journals logical entries for changes in metadata.

As shown in Table 1, from the perspective of error reporting and fsync behavior, XFS is similar to that of ext4 ordered mode. Specifically, failing to write data blocks leads to fsync failure and the faulty data pages are marked clean even though they contain new data that has not been propagated to disk; as a result, applications that read this faulty data will see the new data only until the page has been evicted from the page cache. Similarly, failing to write a journal block will cause fsync failure, while failing to write a metadata block will not. XFS remains available for reads and writes after data-block faults.

XFS handles fsync failures in a few ways that are different than ext4 ordered mode. First, on a journal-block fault, XFS shuts down the file system entirely instead of merely remounting in read-only mode; thus, all subsequent read and write operations fail. Second, XFS retries metadata writes when it encounters a fault during checkpointing; the retry limit is determined by a value in /sys/fs/xfs/*/error/metadata/*/max_retries, but is infinite by default. If the retry limit is exceeded, XFS again shuts down the file system.

The multi-block append workload illustrates how XFS handles metadata when writes to related data blocks fail. If the write to the first data block fails, XFS writes no metadata to the journal and fails the fsync immediately. When later data blocks are successfully appended to this file, the metadata is updated which creates a non-overwritten block in the file corresponding to the first write. If instead, a write to a

data block contained in the last journal transaction fails, the on-disk metadata is not updated to reflect any of these last writes (i.e., the size of the file is not increased if any related blocks fail in the last transaction). [3] Thus, while in ext4 a failed write always causes a non-overwritten block, in XFS, non-overwritten blocks cannot exist at the end of a file. However, for either file system, if the failed blocks remain in the page cache, applications can read those blocks regardless of whether they are in the middle or the end of a file.

3.3.3 Btrfs

Btrfs is a copy-on-write file system that avoids writing to the same block twice except for the superblock which contains root-node information. At a high level, some of the actions in Btrfs are similar to those in a journaling file system: instead of writing to a journal, Btrfs writes to a log tree to record changes when an `fsync` is performed; instead of checkpointing to fixed in-place locations, Btrfs writes to new locations and updates the roots in its superblock. However, since Btrfs is based on copy-on-write, it has a number of interesting differences in how it handles `fsync` failures compared to ext4 and XFS, as shown in Table 1.

Like ext4 ordered mode and XFS, Btrfs fails `fsync` when it encounters data-block faults. However, unlike ext4 and XFS, Btrfs effectively reverts the contents of the data block (and any related metadata) back to its old state (and marks the page clean). Thus, if an application reads the data after this failure, it will never see the failed operation as a temporary state. As in the other file systems, Btrfs remains available after this data-block fault.

Similar to faults to the journal in the other file systems, in Btrfs, faults to the log-tree result in a failed `fsync` and a remount in read-only mode. Unlike ext4 and XFS, faults in the metadata blocks during checkpointing result in a remount in read-only mode (but `fsync` still does not return an error).

The multi-block append workload illustrates interesting behavior in Btrfs block allocation. If the first append fails, the state of the file system, including the B-tree that tracks all free blocks, is reverted. However, the next append will continue to write at the (incorrectly) updated offset stored in the file descriptor, creating a hole in the file. Since the state of the B-tree was reverted, the deterministic block allocator will choose to allocate the same block again for the next append operation. Thus, if the fault to that particular block was transient, the next `write` and `fsync` will succeed and there will simply be a one block hole in the file. If the fault to that particular block occurs multiple times, future writes will continue to fail; as a result, Btrfs may cause more holes within a file than ext4 and XFS. However, unlike ext4 and XFS, the file does not have block overwrite failures.

[3] To be precise, the mtime and ctime of the file are updated, but not the size of the file. Additional experiments removed for space confirm this behavior.

3.3.4 File System Summary

We now present a set of observations for the file systems based on the questions from Section §3.2.3.

File System Behavior to `fsync` Failures. On all the three file systems, only data and journal-block failures lead to `fsync` failures (Q1). Metadata-block failures do not result in `fsync` failures as metadata blocks are written to the journal during an `fsync`. However, during a checkpoint, any metadata failure on XFS and Btrfs lead to unavailability (Q8) while ext4 logs the error and continues.[4]

On both modes of ext4 and XFS, metadata is persisted even after the file system encounters a data-block failure (Q2); timestamps are always updated in both the file systems. Additionally, ext4 appends a new block to the file and updates the file size while XFS does so only when followed by a future successful `fsync`. As a result, we find non-overwritten blocks in both the middle and end of files for ext4, but in only the middle for XFS (Q10). Btrfs does not persist metadata after a data-block failure. However, because the process file-descriptor offset is incremented, future `writes` and `fsyncs` cause a hole in the middle of the file (Q10).

Among the three, XFS is the only file system that retries metadata-block writes. However, none of them retry data or journal-block writes (Q3).

All the file systems mark the page clean even after `fsync` fails (Q4). In both modes of ext4 and XFS, the page contains the latest write while Btrfs reverts the in-memory state to be consistent with what is on disk (Q5).

We note that even though all the file systems mark the page clean, this is not due to any behavior inherited from the VFS layer. Each file system registers its own handlers to write pages to disk (`ext4_writepages`, `xfs_vm_writepages`, and `btrfs_writepages`). However, each of these handlers call `clear_page_dirty_for_io` before submitting the bio request and do not set the dirty bit in case of failure in order to avoid memory leaks[5], replicating the problem independently.

Failure Reporting. While all file systems report data-block failures by failing `fsync`, ext4 ordered mode, XFS, and Btrfs fail the immediate `fsync`. As ext4 data mode puts data in the journal, the first `fsync` succeeds and the next `fsync` fails. (Q6). All block write failures, irrespective of block type are logged in the syslog (Q7).

After Effects. Journal block failures always lead to file-system unavailability. On XFS and Btrfs, metadata-block failures do so as well (Q8). While ext4 and Btrfs remount in read-only mode, XFS shuts down the file system (Q9). Holes and non-overwritten blocks (Q10) have been covered previously as part of Q2.

Recovery. None of the file systems alert the user to run a

[4] Ext4's error handling behavior for metadata has unintended side-effects but we omit the results as the rest of the paper focuses on data-block failures.

[5] Ext4 focuses on the common case of users removing USB sticks while still in use. Dirty pages that can never be written to the removed USB stick have to be marked clean to unmount the file system and reclaim memory [23].

file-system checker. However, the Btrfs checker is capable of detecting holes in files (Q11).

4 Application Study

We now focus on how applications are affected by `fsync` failures. In this section, we first describe our fault model with CuttleFS, followed by a description of the workloads, execution environment, and the errors we look for. Then, we present our findings for five widely used applications: Redis (v5.0.7), LMDB (v0.9.24), LevelDB (v1.22), SQLite (v3.30.1), and PostgreSQL (v12.0).

4.1 CuttleFS

We limit our study to how applications are affected by data-block failures as journal-block failures lead to unavailability and metadata-block failures do not result in `fsync` failures (§3.3). Our fault model is simple: when an application writes data, we inject a single fault to a data block or a sector within it.

We build CuttleFS[6] - a FUSE [39] file system to emulate the different file-system reactions to failures defined by our fault model. Instead of using the kernel's page cache, CuttleFS maintains its own page cache in user-space memory. Write operations modify user-space pages and mark them dirty while read operations serve data from these pages. When an application issues an `fsync` system call, CuttleFS synchronizes data with the underlying file system.

CuttleFS has two modes of operation: trace mode and fault mode. In trace mode, CuttleFS tracks writes and identifies which blocks are eventually written to disk. This is different from just tracing a `write` system call as an application may write to a specific portion of a file multiple times before it is actually flushed to disk.

In fail mode, CuttleFS can be configured to fail the ith write to a sector or block associated with a particular file. On `fsync` failure, as CuttleFS uses in-memory buffers, it can be directed to mark a page clean or dirty, keep the latest content, or revert the file to the previous state. Error reporting behavior can be configured to report failures immediately or on the next `fsync` call. In short, CuttleFS can react to `fsync` failures in any of the ways mentioned in Table 1 (Q4,5,6). Additionally, CuttleFS accepts commands to evict all or specific clean pages.

We configure CuttleFS to emulate the failure reactions of the file systems studied in Section 3.3. For example, in order to emulate ext4 ordered mode and XFS (as they both have similar failure reactions), we configure CuttleFS to mark the page clean, keep the latest content, and report the error immediately. Henceforth, when presenting our findings and referring to characteristics emulated by CuttleFS, we use $\text{CuttleFS}_{\text{ext4o,xfs}}$ for the above configuration. When the page is marked clean, has the latest content, but the error is reported on the next `fsync`, we use $\text{CuttleFS}_{\text{ext4d}}$. When the page is marked clean, the content matches what is on disk, and the error is reported immediately, we refer to it as $\text{CuttleFS}_{\text{btrfs}}$.

[6] Cuttlefish are sometimes referred to as the "chameleons of the sea" because of their ability to rapidly alter their skin color within a second. CuttleFS can change characteristics much faster.

4.2 Workloads and Execution Environment

We run CuttleFS in trace mode and identify which blocks are written to by an application. For each application, we choose a simple workload that inserts a single key-value pair, a commonly used operation in many applications. We perform experiments both with an existing key (update) as well as a new key (insert). The keys can be of size 2B or 1KB.[7] The values can be of size 2B or 12KB. We run experiments for all four combinations. The large keys allow for the possibility of failing a single sector within the key and large values for pages within a value. Since SQLite and PostgreSQL are relational database management systems, we create a single table with two columns: keys and values.

Using the trace, we generate multiple failure sequences for each of the identified blocks and sectors within them. We then repeat the experiment multiple times with CuttleFS in fault mode, each time with a different failure sequence and file-system reaction. In order to observe the effects after a fault, we dump all key-value pairs before and after the workload.

We look for the following types of errors when performing the experiments:

- **OldValue (OV):** The system returns the new value for a while but then reverts to an old value, or the system conveys a successful response but returns the old value later on.
- **FalseFailure (FF):** The system informs the user that the operation failed but returns the new value in the future.
- **KeyCorruptions (KC)** and **ValueCorruptions (VC):** Corrupted keys or values are obliviously returned.
- **KeyNotFound (KNF):** The system informs the user that it has successfully inserted a key but it cannot be found later on, or the system fails to update a key to a new value but the old key-value pair disappears as well.

We also identify the factors within the execution environment that cause all these errors to be manifested. If an application maintains its own in-memory data structures, some errors may occur only when an application restarts and rebuilds in-memory state from the file system. Alternatively, the manifestation of these errors may depend on state changes external to the application, such as a single page eviction or a full page cache flush. We encode these different scenarios as:

- **App=KeepGoing:** The application continues without restarting.
- **App=Restart:** The application restarts either after a crash or a graceful shutdown. This forces the application to rebuild in-memory state from disk.

[7] As LMDB limits key sizes to 511B, we use key sizes of 2B and 511B for LMDB experiments.

	A=KeepGoing	A=Restart
BC=Keep	╲	╱
BC=Evict	|	–

Applications		ext4o,xfs = { clean, differs, immediate }					ext4d = { clean, differs, next `fsync` }					btrfs = { clean, matches, immediate }				
		OV	**FF**	**KC**	**VC**	**KNF**	**OV**	**FF**	**KC**	**VC**	**KNF**	**OV**	**FF**	**KC**	**VC**	**KNF**
Redis		–		–	–	–	–		–	–	–	≠				≠
LMDB			–				+			+						
LevelDB			╱				–		+	+	+					
SQLite	Rollback		+		+			+		–	–		✳			
	WAL		╱							–	–					
PostgreSQL	Default		≠				–				–					
	Direct I/O						–				–					

Table 2: **Findings for Applications on `fsync` Failure.** *The table lists the different types of errors that manifest for applications when `fsync` fails due to a data-block write fault. The errors (OV, FF, KC, VC, KNF) are described in §4.2. We group columns depending on how a file system reacts to an `fsync` failure according to our findings in §3.3 for Q4, Q5, and Q6. For example, both ext4 ordered and XFS (ext4o,xfs) mark a page* ***clean****, the page* ***differs*** *in in-memory and on-disk content, and the `fsync` failure is reported* ***immediately****. For each application, we describe when the error manifests, in terms of combinations of the four different execution environment factors (§4.2) whose symbols are provided at the top left corner. For example, OldValue manifests in Redis in the first group (ext4-ordered, XFS) only on (A)App=Restart,(BC)BufferCache=Evict. However, in the last group (Btrfs), the error manifests both on App=Restart,BufferCache=Evict as well as App=Restart,BufferCache=Keep, depicted as a combination of the two symbols.*

- **BufferCache=Keep:** No evictions take place.
- **BufferCache=Evict:** One or more clean pages are evicted.

Note that BufferCache=Evict can manifest by clearing the entire page cache, restarting the file system, or just evicting clean pages due to memory pressure. A full system restart would be the combination of App=Restart and BufferCache=Evict, which causes a loss of both clean and dirty pages in memory while also forcing the application to restart and rebuild state from disk.

Configuring CuttleFS to fail a certain block and react according to one of the file-system reactions while the application runs only addresses App=KeepGoing and BufferCache=Keep. The remaining three scenarios are addressed as follows. To simulate App=Restart and BufferCache=Keep, we restart the application and dump all key-value pairs, ensuring that no page in CuttleFS is evicted. To address the remaining two scenarios, we instruct CuttleFS to evict clean pages for both App=KeepGoing and App=Restart.

4.3 Findings

We configured all five applications to run in the form that offers most durability and discuss what they are in their respective sections. Table 2 summarizes the per-application results across different failure characteristics.

Note that these results are only for the simple workload that inserts a single key-value pair. A complex workload may exhibit more errors or mask the ones we observe.

Redis: Redis is an in-memory data-structure store, used as a database, cache, and message broker. By default, it periodically snapshots in-memory state to disk. However, for better durability guarantees, it provides options for writing every operation that modifies the store to an append-only file *(aof)* [19] and how often to `fsync` the *aof*. In the event of a crash or restart, Redis rebuilds in-memory state by reading the contents of the *aof*.

We configure Redis to `fsync` the file for every operation, providing strong durability. Thus, whenever Redis receives a request like an insert operation that modifies state, it writes the request to the *aof* and calls `fsync`. However, Redis trusts the file system to successfully persist the data and does not check the `fsync` return code. Regardless of whether `fsync` fails or not, Redis returns a successful response to the client.

As Redis returns a successful response to the client irrespective of `fsync` failure, FalseFailures do not occur. Since Redis reads from disk only when rebuilding in-memory state, errors may occur only during App=Restart.

On $\text{CuttleFS}_{\text{ext4o,xfs}}$ and $\text{CuttleFS}_{\text{ext4d}}$, Redis exhibits OldValue, KeyCorruption, ValueCorruption, and KeyNotFound errors. However, as seen in Table 2, these errors occur only on BufferCache=Evict and App=Restart. On BufferCache=Keep, the page contains the latest write which allows Redis to rebuild the latest state. However, when the page is evicted, future reads will force a read from disk, causing Redis to read whatever is on that block. OldValue and KeyNotFound errors manifest when a fault corrupts the *aof* format. When Redis restarts, it either ignores these entries when scanning the *aof*, or recommends running the *aof checker* which truncates the file to the last non-corrupted entry. A KeyCorruption and ValueCorruption manifest when the fault is within the key or value portion of the entry.

On $\text{CuttleFS}_{\text{btrfs}}$, Redis exhibits OldValue and KeyNotFound errors. These errors occur on App=Restart, regardless of buffer-cache state. When Redis restarts, the entries are missing from the *aof* as the file was reverted, and thus, the insert or update operation is not applied.

LMDB: Lightning Memory-Mapped Database (LMDB) is an embedded key-value store which uses B+Tree data structures whose nodes reside in a single file. The first two pages of the file are metadata pages, each of which contain a transaction ID and the location of the root node. Readers always use the metadata page with the latest transaction ID while writers make changes and update the older metadata page.

LMDB uses a copy-on-write bottom-up strategy [13] for committing write transactions. All new nodes from leaf to root are written to unused or new pages in the file, followed by an `fsync`. An `fsync` failure terminates the operation without updating the metadata page and notifies the user. If `fsync` succeeds, LMDB proceeds to update the old metadata page with the new root location and transaction ID, followed by another `fsync`.[8] If `fsync` fails, LMDB writes an old transaction ID to the metadata page in memory, preventing future readers from reading it.

On CuttleFS$_{ext4o,xfs}$, LMDB exhibits FalseFailures. When LMDB writes the metadata page, it only cares about the transaction ID and new root location, both of which are contained in a single sector. Thus, even though the sector is persisted to disk, failures in the seven other sectors of the metadata page can cause an `fsync` failure. As mentioned earlier, LMDB writes an old transaction ID (say ID1) to the metadata page in memory and reports a failure to the user. However, on BufferCache=Evict and App=Restart (such as a machine crash and restart), ID1 is lost as it was only written to memory and not persisted. Thus, readers read from the latest transaction ID which is the previously failed transaction.

LMDB does not exhibit FalseFailures in CuttleFS$_{ext4d}$ as the immediate successful `fsync` results in a success to the client. Instead, ValueCorruptions and OldValue errors occur on BufferCache=Evict, regardless of whether the application restarts or not. ValueCorruptions occur when a block containing a part of the value experiences a fault. As LMDB *mmaps*() the file and reads directly from the page cache, BufferCache=Evict such as a page eviction leads to reading the value of the faulted block from disk. OldVersion errors occur when the metadata page experiences a fault. The file system responds with a successful `fsync` initially (as data is successfully stored in the ext4 journal). For a short time, the metadata page has the latest transaction ID. However, when the page is evicted, the metadata page reverts to the old transaction ID on disk, resulting in readers reading the old value. KeyCorruptions do not occur as the maximum allowed key size is 511B.

As CuttleFS$_{btrfs}$ reports errors immediately, it does not face the problems seen in CuttleFS$_{ext4d}$. FalseFailures do not occur as the file is reverted to its previous consistent state. We observe this same pattern in many of the applications and omit them from the rest of the discussion unless relevant.

LevelDB: LevelDB is a widely used key-value store based on LSM trees. It stores data internally using MemTables and SSTables [33]. Additionally, LevelDB writes operations to a log file before updating the MemTable. When a MemTable reaches a certain size, it becomes immutable and is written to a new file as an SSTable. SSTables are always created and never modified in place. On a restart, if a log file exists, LevelDB creates an SSTable from its contents.

We configure LevelDB to `fsync` the log after every write, for stronger durability guarantees. If `fsync` fails, the MemTable is not updated and the user is notified about the failure. If `fsync` fails during SSTable creation, the operation is cancelled and the SSTable is left unused.

On CuttleFS$_{ext4o,xfs}$, as seen in Table 2, LevelDB exhibits FalseFailures only on App=Restart with BufferCache=Keep. When LevelDB is notified of `fsync` failure to the log file, the user is notified of the failure. However, on restart, since the log entry is in the page cache, LevelDB includes it while creating an SSTable from the log file. Read operations from this point forward return the new value, reflecting FalseFailures. FalseFailures do not occur on BufferCache=Evict as LevelDB is able to detect invalid entries through CRC checksums [33]. Faults in the SSTable are detected immediately and do not cause any errors as the newly generated SSTable is not used by LevelDB in case of a failure.

On CuttleFS$_{ext4d}$, LevelDB exhibits KeyNotFound and OldVersion errors when faults occur in the log file. When inserting a key-value pair, `fsync` returns successfully, allowing future read operations to return the new value. However, on BufferCache=Evict and App=Restart, LevelDB rejects the corrupted log entry and returns the old value for future read operations. Depending on whether we insert a new or existing key, we observe KeyNotFound or OldVersion errors when the log entry is rejected. Additionally, LevelDB exhibits KeyCorruption, ValueCorruption, and KeyNotFound errors for faults that occur in the SSTables. Ext4 data mode may only place the data in the journal and return a successful `fsync`. Later, during checkpointing, the SSTable is corrupted due to the fault. These errors manifest only on BufferCache=Evict, either while the application is running or on restart, depending on when the SSTable is read from disk.

SQLite: SQLite is an embedded RDBMS that uses BTree data structures. A separate BTree is used for each table and index but all BTrees are stored in a single file on disk, called the "main database file" *(maindb)*. During a transaction, SQLite stores additional information in a second file called the "rollback journal" *(rj)* or the "write-ahead log" *(wal)* depending on which mode it is operating in. In the event of a crash or restart, SQLite uses these files to ensure that committed or rolled-back transactions are reflected in the *maindb*. Once a transaction completes, these files are deleted. We perform experiments for both modes.

SQLite RollBack: In rollback journal mode, before SQLite modifies its user-space buffers, it writes the original contents to the *rj*. On commit, the *rj* is `fsyncd`. If it succeeds, SQLite writes a header to the *rj* and `fsyncs` again (2 `fsyncs` on the *rj*). If a fault occurs at this point, only the state in the user-space buffers need to be reverted. If not, SQLite proceeds to write to the *maindb* so that it reflects the state of the user-space buffers. *maindb* is then `fsyncd`. If the `fsync`

[8] To be precise, LMDB does not do a write followed by an `fsync` for metadata page updates. Instead, it uses a file descriptor that is opened in O_SYNC mode. On a write, only the metadata page is flushed to disk. On failure, it uses a normal file descriptor.

fails, SQLite needs to rewrite the old contents to the *maindb* from the *rj* and revert the state in its user-space buffers. After reverting the contents, the *rj* is deleted.

On CuttleFS$_{ext4o,xfs}$, SQLite Rollback exhibits FalseFailures and ValueCorruptions on BufferCache=Evict, regardless of whether the application restarts or not. When faults occur in the *rj*, SQLite chooses to revert in-memory state using the *rj* itself as it contains just enough information for a rollback of the user-space buffers. This approach works well as long as the latest contents are in the page cache. However, on BufferCache=Evict, when SQLite reads the *rj* to rollback in-memory state, the *rj* does not contain the latest write. As a result, SQLite's user-space buffers can still have the new contents (FalseFailure) or a corrupted value, depending on where the fault occurs.

SQLite Rollback exhibits FalseFailures in CuttleFS$_{ext4d}$ for the same reasons mentioned above as the `fsync` failure is caught on the second `fsync` to the *rj*. Additionally, due to the late error reporting in CuttleFS$_{ext4d}$, SQLite Rollback exhibits ValueCorruption and KeyNotFound errors when faults occur in the *maindb*. SQLite sees a successful `fsync` after writing data to the *maindb* and proceeds to delete the *rj*. However, on App=Restart and BufferCache=Evict, the above mentioned errors manifest depending on where the fault occurs.

On CuttleFS$_{btrfs}$, SQLite Rollback exhibits FalseFailures for the same reasons mentioned above. However, they occur irrespective of whether buffer-cache state changes due to the fact that the contents in the *rj* are reverted. As there is no data in the *rj* to recover from, SQLite leaves the user-space buffers untouched. ValueCorruptions cannot occur as no attempt is made to revert the in-memory content.

SQLite WAL: Unlike SQLite Rollback, changes are written to a write-ahead log *(wal)* on a transaction commit. SQLite calls `fsync` on the *wal* and proceeds to change in-memory state. If `fsync` fails, SQLite immediately returns a failure to the user. If SQLite has to restart, it rebuilds state from the *maindb* first and then changes state according to the entries in the *wal*. To ensure that the *wal* does not grow too large, SQLite periodically runs a Checkpoint Operation to modify *maindb* with the contents from the *wal*.

On CuttleFS$_{ext4o,xfs}$, as seen in Table 2, SQLite WAL exhibits FalseFailures only on App=Restart with BufferCache=Keep, for reasons similar to LevelDB. It reads valid log entries from the page cache even though they might be invalid due to faults on disk.

On CuttleFS$_{ext4d}$, SQLite WAL exhibits ValueCorruption and KeyNotFound Errors when there are faults in the *maindb* during a Checkpoint Operation for the same reasons mentioned in SQLite Rollback.

PostgreSQL: PostgreSQL is an object-relational database system that maintains one file per database table. On startup, it reads the on-disk tables and populates user-space buffers. Similar to SQLite WAL, PostgreSQL reads entries from the write-ahead log *(wal)* and modifies user-space buffers accordingly. Similar to SQLite WAL, PostgreSQL runs a checkpoint operation, ensuring that the *wal* does not grow too large. We evaluate two configurations of PostgreSQL: the default configuration and a DirectIO configuration.

PostgreSQL Default: In the default mode, PostgreSQL treats the *wal* like any other file, using the page cache for reads and writes. PostgreSQL notifies the user of a successful *commit* operation only after an `fsync` on the *wal* succeeds. During a checkpoint, PostgreSQL writes data from its user-space buffers into the table and calls `fsync`. If the `fsync` fails, PostgreSQL, aware of the problems with `fsync` [8], chooses to crash. Doing so avoids truncating the *wal* and ensures that checkpointing can be retried later.

On CuttleFS$_{ext4o,xfs}$, PostgreSQL exhibits FalseFailures for reasons similar to LevelDB. While App=Restart is necessary to read the entry from the log, BufferCache=Evict is not. Further, the application restart cannot be avoided as PostgreSQL intentionally crashes on an `fsync` failure. On BufferCache=Keep, PostgreSQL reads a valid log entry in the page cache. On BufferCache=Evict, depending on which block experiences the fault, PostgreSQL either accepts or rejects the log entry. FalseFailures manifest when PostgreSQL accepts the log entry. However, if the file system were to also crash and restart, the page cache would match the on-disk state, causing PostgreSQL to reject the log entry. Unfortunately, ext4 currently does not behave as expected with mount options `data_err=abort` and `errors=remount-ro` (§3.3.1).

Due to the late error reporting in CuttleFS$_{ext4d}$, as seen in Table 2, PostgreSQL exhibits OldVersion and KeyNotFound Errors when faults occur in the database table files. As PostgreSQL maintains user-space buffers, these errors manifest only on BufferCache=Evict with App=Restart. During a checkpoint operation, PostgreSQL writes the user-space buffers to the table. As the fault is not yet reported, the operation succeeds and the *wal* is truncated. If the page corresponding to the fault is evicted and PostgreSQL restarts, it will rebuild its user-space buffers using an incorrect on-disk table file. The errors are exhibited depending on where the fault occurs. While KeyNotFound errors occur in other applications when a new key is inserted, PostgreSQL *loses existing keys on updates* as it modifies the table file in-place.

PostgreSQL DIO: In the DirectIO mode, PostgreSQL bypasses the page cache and writes to the *wal* using DirectIO. The sequence of operations during a transaction commit and a checkpoint are exactly the same as the default mode.

FalseFailures do not occur as the page cache is bypassed. However, OldVersion and KeyNotFound errors still occur in CuttleFS$_{ext4d}$ for the same reasons mentioned above as writes to the database table files do not use DirectIO.

5 Discussion

We now present a set of observations and lessons for handling `fsync` failures across file systems and applications.

#1: Existing file systems do not handle fsync failures uniformly. In an effort to hide cross-platform differences, POSIX is intentionally vague on how failures are handled. Thus, different file systems behave differently after an `fsync` failure (as seen in Table 1), leading to non-deterministic outcomes for applications that treat all file systems equally. *We believe that the POSIX specification for `fsync` needs to be clarified and the expected failure behavior described in more detail.*

#2: Copy-on-Write file systems such as Btrfs handle fsync failures better than existing journaling file systems like ext4 and XFS. Btrfs uses new or unused blocks when writing data to disk; the entire file system moves from one state to another on success and no in-between states are permitted. Such a strategy defends against corruptions when only some blocks contain newly written data. *File systems that use copy-on-write may be more generally robust to `fsync` failures than journaling file systems.*

#3: Ext4 data mode provides a false sense of durability. Application developers sometimes choose to use a data journaling file system despite its lower performance because they believe data mode is more durable [11]. Ext4 data mode does ensure data and metadata are in a "consistent state", but only from the perspective of the file system. As seen in Table 2, application-level inconsistencies are still possible. Furthermore, applications cannot determine whether an error received from `fsync` pertains to the most recent operation or an operation sometime in the past. *When failed intentions are a possibility, applications need a stronger contract with the file system, notifying them of relevant context such as data in the journal and which blocks were not successfully written.*

#4: Existing file-system fault-injection tests are devoid of workloads that continue to run post failure. While all file systems perform fault-injection tests, they are mainly to ensure that the file system is consistent after encountering a failure. Such tests involve shutting down the file system soon after a fault and checking if the file system recovers correctly when restarted. *We believe that file-system developers should also test workloads that continue to run post failure, and see if the effects are as intended.* Such effects should then be documented. File-system developers can also quickly test the effect on certain characteristics by running those workloads on CuttleFS before changing the actual file system.

#5: Application developers write OS-specific code, but are not aware of all OS-differences. The FreeBSD VFS layer chooses to re-dirty pages when there is a failure (except when the device is removed) [6] while Linux hands over the failure handling responsibility to the individual file systems below the VFS layer (§3.3.4). *We hope that the Linux file-system maintainers will adopt a similar approach in an effort to handle `fsync` failures uniformly across file systems.* Note that it is also important to think about when to classify whether a device has been removed. For example, while storage devices connected over a network aren't really as permanent as local hard disks, they are more permanent than removable USB sticks. Temporary disconnects over a network need not be perceived as device removal and re-attachment; pages associated with such a device can be re-dirtied on write failure.

#6: Application developers do not target specific file systems. We observe that data-intensive applications configure their durability and error-handling strategies according to the OS they are running on, but treat all file systems on a specific operating system equally. Thus, as seen in Table 2, a single application can manifest different errors depending on the file system. *If the POSIX standard is not refined, applications may wish to handle `fsync` failures on different file systems differently.* Alternatively, applications may choose to code against *failure handling characteristics* as opposed to specific file systems, but this requires file systems to expose some interface to query characteristics such as "Post Failure Page State/Content" and "Immediate/Delayed Error Reporting".

#7: Applications employ a variety of strategies when fsync fails, but none are sufficient. As seen in Section 4.3, Redis chooses to trust the file system and does not even check `fsync` return codes, LMDB, LevelDB, and SQLite revert in-memory state and report the error to the application while PostgreSQL chooses to crash. We have seen that none of the applications retry `fsync` on failure; application developers appear to be aware that pages are marked clean on `fsync` failure and another `fsync` will not flush additional data to disk. Despite the fact that applications take great care to handle a range of errors from the storage stack (e.g., LevelDB writes CRC Checksums to detect invalid log entries and SQLite updates the header of the rollback journal only after the data is persisted to it), data durability cannot be guaranteed as long as `fsync` errors are not handled correctly. *While no one strategy is always effective, the approach currently taken by PostgreSQL to use direct IO may best handle `fsync` failures.* If file systems do choose to report failure handling characteristics in a standard format, applications may be able to employ better strategies. For example, applications can choose to keep track of dirtied pages and re-dirty them by reading and writing back a single byte if they know that the page content is not reverted on failure (ext4, XFS). On Btrfs, one would have to keep track of the page as well as its content. For applications that access multiple files, it is important to note that the files can exist on different file systems.

#8: Applications run recovery logic that accesses incorrect data in the page cache. Applications that depend on the page cache for faster recovery are susceptible to FalseFailures. As seen in LevelDB, SQLite, and PostgreSQL, when the *wal* incurs an `fsync` failure, the applications fail the operation and notify the user; In these cases, while the on-disk state may be corrupt, the entry in the page cache is valid; thus, an application that recovers state from the *wal* might read partially valid entries from the page cache and incorrectly update on-disk state. *Applications should read the on-disk content of files when performing recovery.*

#9: Application recovery logic is not tested with low level block faults. Applications test recovery logic and possibilities of data loss by either mocking system call return codes or emulating crash-restart scenarios, limiting interaction with the underlying file system. As a result, failure handling logic by the file system is not exercised. *Applications should test recovery logic using low-level block injectors that force underlying file-system error handling.* Alternatively, they could use a fault injector like CuttleFS that mimics different file-system error-handling characteristics.

6 Related Work

In this section, we discuss how our work builds upon and differs from past studies in key ways. We include works that study file systems through fault injection, error handling in file systems, and the impact of file-system faults on applications.

Our study on how file systems react to failures is related to work done by Prabhakaran et al. with IRON file systems [49] and a more recent study conducted by Jaffer et al. [40]. Other works study specific file systems such as NTFS [28] and ZFS [58]. All these studies inject failures beneath the file system and analyze if and how file systems detect and recover from them. These studies use system-call workloads (e.g., writes and reads) that make the file system interact with the underlying device.

While prior studies do exercise some portions of the `fsync` path through single system-call operations, they do not exercise the checkpoint path. More importantly, in contrast to these past efforts, our work focuses specifically on the *in-memory* state of a file system and the effects of *future operations* on a file system that has encountered a write fault. Specifically, in our work, we choose workloads that continue after a fault has been introduced. Such workloads help in understanding the after-effects of failures during `fsync` such as masking of errors by future operations, fixing the fault, or exacerbating it.

Mohan et al. [45] use bounded black-box crash testing to exhaustively generate workloads and discover many crash-consistency bugs by simulating power failures at different persistence points. Our work focuses on transient failures that may not necessarily cause a file system to crash and the effect on applications even though a file system may be consistent. Additionally, we inject faults in the middle of an `fsync` as opposed to after a successful `fsync` (persistence point).

Gunawi et al. describe the problem of failed intentions [36] in journaling file systems and suggest chained transactions to handle such faults during checkpointing. Another work develops a static-analysis technique named Error Detection and Propagation [37] and conclude that file systems neglect many write errors. Even though the Linux kernel has improved its block-layer error handling [10], file systems may still neglect write errors. Our results are purely based on injecting errors in bio requests that the file system can detect.

Vondra describes how certain assumptions about `fsync` behavior led to data loss in PostgreSQL [55]. The data loss behavior was reproduced using a device mapper with the dm-error target which inspired us to build our own fault injector (dm-loki [4]) atop the device mapper, similar to dm-inject [40]. Additionally, the FSQA suite (xfstests) [7] emulates write errors using the dm-flakey target [5]. While dm-flakey is useful for fault-injection testing, faults are injected based on current time; the device is available for x seconds and then exhibits unreliable behavior for y seconds (x and y being configurable). Furthermore, any change in configuration requires suspending the device. To increase determinism and avoid relying on time, dm-loki injects faults based on access patterns (e.g., fail the 2nd and 4th write to block 20) and is capable of accepting configuration changes without device suspension.

Recent work has shifted the focus to study the effects of file-system faults in distributed storage systems [34] and high-performance parallel systems [29]. Similarly, our work focuses on understanding how file systems and applications running on top of them behave in the presence of failures.

7 Conclusions

We show that file systems behave differently on `fsync` failure. Application developers can only assume that the underlying file system experienced a fault and that data may have either been persisted partially, completely, or not at all. We show that applications assuming more than the above are susceptible to data loss and corruptions. The widely perceived crash-restart fix in the face of `fsync` failures does not always work; applications recover incorrectly due to on-disk and in-memory mismatches.

However, we believe that applications can provide stronger guarantees if file systems are more uniform in their failure handling and error reporting strategies. Applications that care about durability should include sector- or block-level fault-injection tests to effectively test recovery code paths. Alternatively, such applications can choose to use CuttleFS to inject faults and mimic file system failure reactions.

We have open sourced CuttleFS at `https://github.com/WiscADSL/cuttlefs` along with the device-mapper kernel module and experiments to reproduce the results in this paper.

8 Acknowledgements

We thank Peter Macko (our shepherd), the anonymous reviewers of ATC '20, and the members of ADSL for their insightful comments and suggestions. We thank CloudLab [32] for providing a great environment to run our experiments. We also thank our sponsors: VMWare, NetApp, and Intel, for their generous support. This material was also supported by funding from NSF grants CNS-1421033, CNS-1763810 and CNS-1838733, and DOE grant DE-SC0014935. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of NSF, DOE, or any other institutions.

References

[1] Atomic Commit In SQLite. https://www.sqlite.org/atomiccommit.html.

[2] Bug-207729 Mounting EXT4 with data_err=abort does not abort journal on data block write failure. https://bugzilla.kernel.org/show_bug.cgi?id=207729.

[3] Bug-27805553 HARD ERROR SHOULD BE REPORTED WHEN FSYNC() RETURN EIO. https://github.com/mysql/mysql-server/commit/8590c8e12a3374eeccb547359750a9d2a128fa6a.

[4] Custom Fault Injection Device Mapper Target: dm-loki. https://github.com/WiscADSL/dm-loki.

[5] Device Mapper: dm-flakey. https://www.kernel.org/doc/html/latest/admin-guide/device-mapper/dm-flakey.html.

[6] FreeBSD VFS Layer re-dirties pages after failed block write. https://github.com/freebsd/freebsd/blob/0209fe3398be56e5e042c422a96a4fbc654247f4/sys/kern/vfs_bio.c#L2646.

[7] FSQA (xfstests). https://git.kernel.org/pub/scm/fs/xfs/xfstests-dev.git/about/.

[8] Fsync Errors - PostgreSQL wiki. https://wiki.postgresql.org/wiki/Fsync_Errors.

[9] fsync(2) - Linux Programmer's Manual. http://man7.org/linux/man-pages/man2/fdatasync.2.html.

[10] Improved block-layer error handling. https://lwn.net/Articles/724307/.

[11] Is data=journal safer for Ext4 as opposed to data=ordered? https://unix.stackexchange.com/q/127235.

[12] LevelDB. https://github.com/google/leveldb.

[13] Lightning Memory-Mapped Database Manager (LMDB). http://www.lmdb.tech/doc/.

[14] POSIX Specification for fsync. https://pubs.opengroup.org/onlinepubs/9699919799/functions/fsync.html.

[15] PostgreSQL. https://www.postgresql.org/.

[16] PostgreSQL: Write-Ahead Logging (WAL). https://www.postgresql.org/docs/current/wal-intro.html.

[17] PostgreSQL's handling of fsync() errors is unsafe and risks data loss at least on XFS . https://www.postgresql.org/message-id/flat/CAMsr%2BYHh%2B5Oq4xziwwoEfhoTZgr07vdGG%2Bhu%3D1adXx59aTeaoQ%40mail.gmail.com.

[18] Redis. https://redis.io/.

[19] Redis Persistence. https://redis.io/topics/persistence.

[20] SQLite. https://www.sqlite.org/index.html.

[21] SQLite Write-Ahead Logging. https://www.sqlite.org/wal.html.

[22] SystemTap. https://sourceware.org/systemtap/.

[23] Why does ext4 clear the dirty bit on I/O error? https://www.postgresql.org/message-id/edc2e4d5-5446-e0db-25da-66db6c020cc3%40commandprompt.com.

[24] WT-4045 Don't retry fsync calls after EIO failure. https://github.com/wiredtiger/wiredtiger/commit/ae8bccce3d8a8248afa0e4e0cf67674a43dede96.

[25] Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*. Arpaci-Dusseau Books, 1.00 edition, August 2018.

[26] Lakshmi N. Bairavasundaram, Garth Goodson, Bianca Schroeder, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. An Analysis of Data Corruption in the Storage Stack. In *Proceedings of the 6th USENIX Symposium on File and Storage Technologies (FAST '08)*, pages 223–238, San Jose, CA, February 2008.

[27] Lakshmi N. Bairavasundaram, Garth R. Goodson, Shankar Pasupathy, and Jiri Schindler. An Analysis of Latent Sector Errors in Disk Drives. In *Proceedings of the 2007 ACM SIGMETRICS Conference on Measurement and Modeling of Computer Systems (SIGMETRICS '07)*, pages 289–300, San Diego, CA, June 2007.

[28] Lakshmi N. Bairavasundaram, Meenali Rungta, Nitin Agrawal, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau, and Michael M. Swift. Analyzing the Effects of Disk-Pointer Corruption. In *Proceedings of the International Conference on Dependable Systems and Networks (DSN '08)*, pages 502–511, Anchorage, Alaska, June 2008.

[29] Jinrui Cao, Om Rameshwar Gatla, Mai Zheng, Dong Dai, Vidya Eswarappa, Yan Mu, and Yong Chen. PFault: A General Framework for Analyzing the Reliability of High-Performance Parallel File Systems. In *Proceedings of the 2018 International Conference on Supercomputing*, pages 1–11, Beijing, China, June 2018.

[30] Vijay Chidambaram, Thanumalayan Sankaranarayana Pillai, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. Optimistic Crash Consistency. In *Proceedings of the 24th ACM Symposium on Operating Systems Principles (SOSP '13)*, pages 228–243, Farmington, PA, November 2013.

[31] Vijay Chidambaram, Tushar Sharma, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. Consistency Without Ordering. In *Proceedings of the 10th USENIX Symposium on File and Storage Technologies (FAST '12)*, pages 101–116, San Jose, CA, February 2012.

[32] Dmitry Duplyakin, Robert Ricci, Aleksander Maricq, Gary Wong, Jonathon Duerig, Eric Eide, Leigh Stoller, Mike Hibler, David Johnson, Kirk Webb, Aditya Akella, Kuangching Wang, Glenn Ricart, Larry Landweber, Chip Elliott, Michael Zink, Emmanuel Cecchet, Snigdhaswin Kar, and Prabodh Mishra. The Design and Operation of CloudLab. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, pages 1–14, Renton, WA, July 2019.

[33] Christian Forfang. Evaluation of High Performance Key-Value Stores. Master's thesis, Norwegian University of Science and Technology, June 2014.

[34] Aishwarya Ganesan, Ramnatthan Alagappan, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. Redundancy Does Not Imply Fault Tolerance: Analysis of Distributed Storage Reactions to Single Errors and Corruptions. In *Proceedings of the 15th USENIX Conference on File and Storage Technologies (FAST '17)*, pages 149–165, Santa Clara, CA, February 2017.

[35] Gregory R. Ganger and Yale N. Patt. Metadata Update Performance in File Systems. In *Proceedings of the 1st Symposium on Operating Systems Design and Implementation (OSDI '94)*, pages 49–60, Monterey, CA, November 1994.

[36] Haryadi S. Gunawi, Vijayan Prabhakaran, Swetha Krishnan, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. Improving File System Reliability with I/O Shepherding. In *Proceedings of the 21st ACM Symposium on Operating Systems Principles (SOSP '07)*, pages 293–306, Stevenson, WA, October 2007.

[37] Haryadi S. Gunawi, Cindy Rubio-González, Remzi H. Arpaci-Dusseau Andrea C. Arpaci-Dusseau, and Ben Liblit. EIO: Error Handling is Occasionally Correct. In *Proceedings of the 6th USENIX Symposium on File and Storage Technologies (FAST '08)*, pages 207–222, San Jose, CA, February 2008.

[38] Robert Hagmann. Reimplementing the Cedar File System Using Logging and Group Commit. In *Proceedings of the 11th ACM Symposium on Operating Systems Principles (SOSP '87)*, pages 155–162, Austin, Texas, November 1987.

[39] FUSE (Filesystem in Userspace). The reference implementation of the Linux FUSE (Filesystem in Userspace) interface. https://github.com/libfuse/libfuse.

[40] Shehbaz Jaffer, Stathis Maneas, Andy Hwang, and Bianca Schroeder. Evaluating File System Reliability on Solid State Drives. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, pages 783–797, Renton, WA, July 2019.

[41] Hannu H. Kari. *Latent Sector Faults and Reliability of Disk Arrays*. PhD thesis, Helsinki University of Technology, September 1997.

[42] Andrew Krioukov, Lakshmi N. Bairavasundaram, Garth R. Goodson, Kiran Srinivasan, Randy Thelen, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. Parity Lost and Parity Regained. In *Proceedings of the 6th USENIX Symposium on File and Storage Technologies (FAST '08)*, pages 127–141, San Jose, CA, February 2008.

[43] Avantika Mathur, Mingming Cao, and Andreas Dilger. Ext4: The Next Generation of the Ext3 File System. *Usenix Association*, 32(3):25–30, June 2007.

[44] Jeffrey C. Mogul. A Better Update Policy. In *Proceedings of the USENIX Summer Technical Conference (USENIX Summer '94)*, pages 99–111, Boston, MA, June 1994.

[45] Jayashree Mohan, Ashlie Martinez, Soujanya Ponnapalli, Pandian Raju, and Vijay Chidambaram. Finding Crash-Consistency Bugs with Bounded Black-Box Crash Testing. In *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI '18)*, pages 33–50, Carlsbad, CA, October 2018.

[46] Thanumalayan Sankaranarayana Pillai, Ramnatthan Alagappan, Lanyue Lu, Vijay Chidambaram, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. Application Crash Consistency and Performance with CCFS. In *Proceedings of the 15th USENIX Conference on File and Storage Technologies (FAST '17)*, pages 181–196, Santa Clara, CA, February 2017.

[47] Thanumalayan Sankaranarayana Pillai, Vijay Chidambaram, Ramnatthan Alagappan, Samer Al-Kiswany, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. All File Systems Are Not Created Equal: On the Complexity of Crafting Crash-Consistent Applications. In *Proceedings of the 11th Symposium on Operating Systems Design and Implementation (OSDI '14)*, pages 433–448, Broomfield, CO, October 2014.

[48] Vijayan Prabhakaran, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. Model-Based Failure Analysis of Journaling File Systems. In *Proceedings of the International Conference on Dependable Systems and Networks (DSN '05)*, pages 802–811, Yokohama, Japan, June 2005.

[49] Vijayan Prabhakaran, Lakshmi N. Bairavasundaram, Nitin Agrawal, Haryadi S. Gunawi, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. IRON File Systems. In *Proceedings of the 20th ACM Symposium on Operating Systems Principles (SOSP '05)*, pages 206–220, Brighton, UK, October 2005.

[50] Ohad Rodeh, Josef Bacik, and Chris Mason. BTRFS: The Linux B-Tree Filesystem. *ACM Transactions on Storage (TOS)*, 9(3):1–32, August 2013.

[51] Bianca Schroeder, Sotirios Damouras, and Phillipa Gill. Understanding Latent Sector Errors and How to Protect Against Them. In *Proceedings of the 8th USENIX Symposium on File and Storage Technologies (FAST '10)*, pages 71–84, San Jose, CA, February 2010.

[52] Margo Seltzer, Peter Chen, and John Ousterhout. Disk Scheduling Revisited. In *Proceedings of the Winter 1990 USENIX Conference*, pages 313–323, Washington, D.C., January 1990.

[53] Chuck Silvers. UBC: An Efficient Unified I/O and Memory Caching Subsystem for NetBSD. In *Proceedings of FREENIX Track: 2000 USENIX Annual Technical Conference*, pages 285–290, San Diego, CA, June 2000.

[54] Adam Sweeney, Doug Doucette, Wei Hu, Curtis Anderson, Mike Nishimoto, and Geoff Peck. Scalability in the XFS File System. In *Proceedings of the USENIX 1996 Annual Technical Conference*, San Diego, CA, January 1996.

[55] Tomas Vondra. PostgreSQL vs. fsync. How is it possible that PostgreSQL used fsync incorrectly for 20 years, and what we'll do about it. Brussels, Belgium, February 2019. https://archive.fosdem.org/2019/schedule/event/postgresql_fsync/.

[56] Youjip Won, Jaemin Jung, Gyeongyeol Choi, Joontaek Oh, Seongbae Son, Jooyoung Hwang, and Sangyeun Cho. Barrier-Enabled IO Stack for Flash Storage. In *Proceedings of the 16th USENIX Conference on File and Storage Technologies (FAST'18)*, pages 211–226, Oakland, CA, February 2018.

[57] Yiying Zhang and Steven Swanson. A Study of Application Performance with Non-Volatile Main Memory. In *Proceedings of the 31st IEEE Conference on Massive Data Storage (MSST '15)*, pages 1–10, Santa Clara, CA, May 2015.

[58] Yupu Zhang, Abhishek Rajimwale, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. End-to-end Data Integrity for File Systems: A ZFS Case Study. In *Proceedings of the 8th USENIX Symposium on File and Storage Technologies (FAST '10)*, pages 29–42, San Jose, CA, February 2010.